Take Home Assessment 2 - Psychrometric Chart

OctaiPipe Frontend Take-Home: Psychrometric SLA Chart

> **Time box:** aim for about 2-3 hours; stop at 4. We prefer deliberate, explained scope cuts to a rushed attempt at everything. If you must choose, prioritise correct chart geometry over visual polish.

Context

ACE monitors data-centre halls against a contractual operating envelope. Envelopes follow the ASHRAE TC 9.9 thermal guidelines and bound the air condition in several quantities at once: dry-bulb temperature, dew point, relative humidity and, at sites with adiabatic cooling, wet-bulb temperature. Operators see this on a psychrometric chart: the envelope is a shaded region and the hall's condition is a moving point. No charting library draws this, so we build it ourselves. This exercise is a scaled-down version of the chart that exists within each ACE recommendation.

The Task

Build a small React + TypeScript app containing a custom psychrometric chart component. Scaffold it however you like. Render with SVG, canvas or low-level scale helpers (d3/visx scales are fine), but not a pre-built chart type. Use the standard psychrometric composition: **dry-bulb temperature (10 to 40 °C) on x** and **humidity ratio (0 to 25 g water per kg dry air) on y**, with RH, wet bulb and dew point overlaid.

Must

1. **Axes and isolines.** Draw the saturation curve (100 % RH) and grey out the region above it; labelled RH curves at 20/40/60/80 %; labelled wet-bulb lines at 5/10/15/20/25 °C; and a **right-hand axis in dew point** (ticks at -10, -5, 0, … 20 °C), since at fixed pressure dew point is a monotonic function of humidity ratio. Clip isolines to the plot and label them where they exit.
2. **Safe zone.** Draw the site `envelope` from `conditions.json` as one shaded region. It is bounded in **four different quantities**, so converting it into a single closed shape on the chart is the heart of the exercise. Absent keys mean "no bound in that direction", so derive the shape from whatever bounds are present; do not hard-code it (we will change the numbers in the follow-up session).
3. **Operating point.** Plot the current reading with a trail of the previous 6. Advance one reading per second through the data source described below.
4. **Compliance.** A reading is compliant when it satisfies every bound (bounds are **inclusive**). Make it obvious when the hall is out and **which bound(s)** are violated.
5. **Hover.** Anywhere on the plot: show dry bulb, humidity ratio, dew point, RH and wet bulb at the cursor. On the point or trail: show that reading's values and timestamp.

6. **Loading and failure.** The data source is intermittently slow and occasionally fails; the UI must show this and never present a stale reading as current without saying so.
7. **Tests.** A few that earn their keep; the conversions and the compliance check are the natural candidates.

Should / could

- Design so a **second site** (own envelope, own readings) is a data change, not a refactor; explain how in the README. `conditions.json` is already `sites: [...]`.
- Warn when a reading is within a small margin of a bound. Optionally draw `allowableEnvelope` as an outer zone with a three-level state.

Not required: responsive layout, accessibility beyond sensible semantics, styling frameworks, persistence, a backend.

Data

`conditions.json` (attached) holds one site with an `envelope` (the ASHRAE recommended envelope plus a site wet-bulb cap), an optional `allowableEnvelope`, and 35 readings of `{ timestamp, dryBulbC, rhPct }` at 5-minute intervals. Dry-bulb bounds are vertical lines on the chart, dew-point bounds horizontal, RH and wet-bulb bounds curves.

Quantity	Lower	Upper	Source
Dry bulb	18 °C	27 °C	ASHRAE recommended
Dew point	-9 °C	15 °C	ASHRAE recommended
Relative humidity	none	60 %	ASHRAE recommended
Wet bulb	none	18.5 °C	Site contract

Consume the readings as though they came from a polled API. Write a small mock module behind an interface such as `getSite(): Promise<Site>` and `nextReading(): Promise<Reading>`, resolving with a short random delay, taking 2 to 3 s roughly every tenth call, and rejecting roughly every fifteenth. When readings run out, stop with a clear "feed ended" state or wrap around; say which and why.

The Physics (Provided)

Temperatures in °C, pressures in kPa, RH as a fraction 0 to 1 (the data file uses **percent**). P = 101.325 kPa.

```
p_ws(T) = 0.61094 * exp(17.625*T / (T + 243.04))      // saturation
vapour pressure
p_w     = RH * p_ws(T)                                  // vapour
pressure
W       = 0.622 * p_w / (P - p_w)                       // humidity
ratio, kg/kg
T_dp    = 243.04*a / (17.625 - a),  a = ln(p_w/0.61094)  // dew point
(inverse Magnus)
p_w     = W * P / (0.622 + W)                           // vapour
pressure from a chart position
RH      = p_w / p_ws(T)                                 // ...then
RH; dew point via the line above

// wet bulb (ASHRAE psychrometric relation)
W = ((2501 - 2.326*T_wb) * W_s(T_wb) - 1.006*(T - T_wb))
    / (2501 + 1.86*T - 4.186*T_wb)
W_s(T_wb) = 0.622 * p_ws(T_wb) / (P - p_ws(T_wb))

```

At fixed RH, W is explicit in T (that is an RH isoline). At fixed T_wb, W is explicit in T (that is a wet-bulb isoline). A fixed dew point is a horizontal line at W(p_ws(T_dp)). Finding T_wb **for a reading** needs a short numerical solve: bisect on T_wb between T_dp and T_db, to about 0.01 °C. Please use this relation rather than an empirical approximation (e.g. Stull), which is only accurate to ±1 °C.

Reference values

Dry bulb	RH	Dew point	Wet bulb	Humidity ratio
25 °C	50 %	13.86 °C	17.89 °C	9.86 g/kg
20 °C	60 %	12.00 °C	15.14 °C	8.71 g/kg
30 °C	30 %	10.54 °C	17.98 °C	7.90 g/kg
18 °C	60 %	10.12 °C	13.41 °C	7.68 g/kg

Deliverables and Logistics

1. A repository (private GitHub repo, or a zip).
2. A README: how to run it and the tests; key decisions and trade-offs, including how you derived the safe-zone shape; where you used AI assistants and anything you rejected or reworked; what you cut and what you would do next.

AI coding assistants are allowed and expected (we use them daily), but you must be able to explain and modify every part of what you submit. Send to **back to the person who sent you the task**; if you need more calendar time, just ask. If we progress, we will spend 30 to 45 minutes walking through your code and extending it together.

What We Look At

- **Chart geometry:** correct scales and isolines; the safe zone derived correctly from all four quantities, with every active bound contributing the right edge.
- **Component design:** scales, physics, layers and data cleanly separated; the chart does not know where readings come from.
- **Data and state:** a sensible update cycle; loading, slow and failure states handled visibly.
- **Code quality and judgement:** readable typed code, a few meaningful tests, and an honest README.